Day 4: Monitoring, Logging & Autoscaling

Welcome to Day 4 – the final day of our Containers & Kubernetes workshop! Today, we focus on **production-ready operations**. You will learn how to:

- Monitor Kubernetes clusters and applications using **Prometheus**.
- Visualise metrics with **Grafana** dashboards.
- Understand logging architecture and basics of the **ELK stack**.
- Automatically scale applications based on load using **Horizontal Pod Autoscaling (HPA)**.
- Implement best practices such as resource requests/limits, health probes, and PodDisruptionBudgets.

By the end of the day, you will have a fully observable and scalable application running on your local Minikube cluster, and you'll be ready to apply these concepts in real-world scenarios.

Prerequisites

- Docker and Minikube running (from previous days).
- `kubectl` configured.
- Helm installed (v3).
- Basic understanding of Kubernetes objects (Deployments, Services).
- A running Minikube cluster with enough resources (recommend at least 4GB RAM, 2 CPUs). You can adjust with:

```
minikube stop
minikube start --cpus=2 --memory=4096
```

09:00 – 10:30 – Conceptual Session: Monitoring & Observability

1.1 Why Monitor Kubernetes?

The Three Pillars of Observability

- **Metrics**: Numerical measurements over time (CPU, memory, request rate, error rate).
- **Logs**: Discrete events with timestamps (application logs, system logs).
- **Traces**: End-to-end request flows across services (distributed tracing).

In Kubernetes, monitoring is essential for:

- **Capacity planning**: Understanding resource usage trends.
- **Anomaly detection**: Spotting unusual behaviour quickly.
- **SLA verification**: Ensuring your application meets performance targets.
- **Troubleshooting**: Correlating metrics and logs to find root causes.

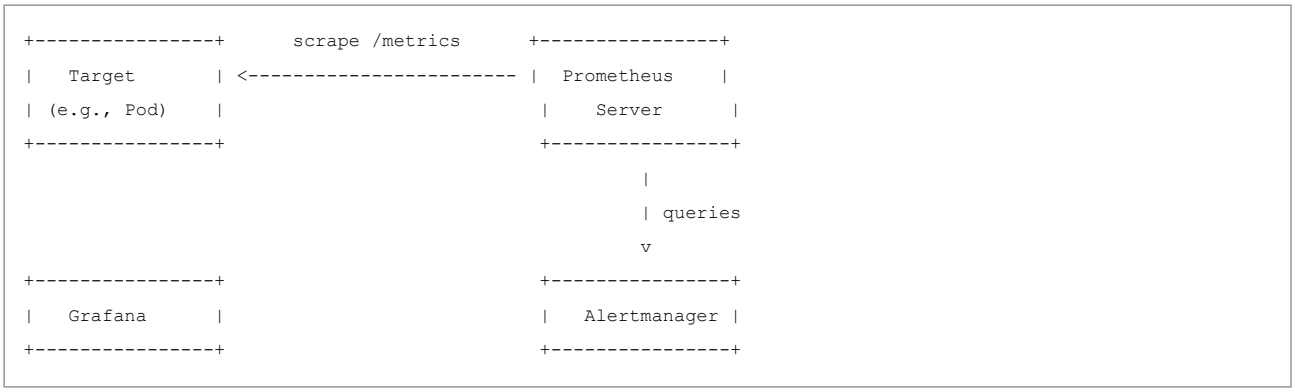
1.2 Prometheus Architecture

Prometheus is a leading open-source monitoring system that scrapes metrics from instrumented targets, stores them in a time-series database, and allows querying with a powerful language called **PromQL**.

Key Components

- **Prometheus server**: Scrapes and stores metrics.
- **Exporters**: Agents that expose metrics from third-party systems (e.g., Node Exporter for host metrics, kube-state-metrics for Kubernetes objects).
- **ServiceMonitor**: A custom resource (when using Prometheus Operator) that defines which services to scrape.
- **Alertmanager**: Handles alerts based on Prometheus rules.

How Prometheus Works (Pull Model)



Prometheus periodically **pulls** metrics from targets over HTTP. Targets must expose a `/metrics` endpoint.

PromQL Basics

- `container_cpu_usage_seconds_total` – counter of CPU seconds used.
- `rate(container_cpu_usage_seconds_total[5m])` – per-second rate averaged over 5 minutes.
- `sum(rate(container_cpu_usage_seconds_total[5m])) by (pod)` – sum rates per pod.

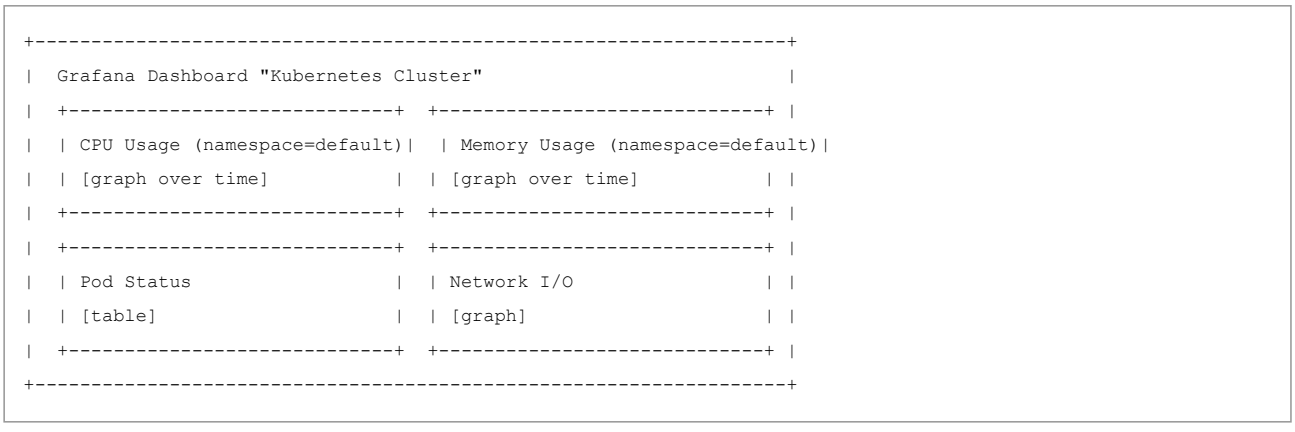
1.3 Grafana Visualization

Grafana is a multi-platform analytics and visualization tool. It connects to Prometheus (and many other data sources) to create dashboards.

Features

- Rich, customizable dashboards with panels (graphs, tables, heatmaps).
- Alerting based on dashboard queries.
- Support for multiple data sources (Prometheus, Elasticsearch, InfluxDB, etc.).

Diagram: Grafana Dashboard Example



1.4 Logging in Kubernetes

Container Logs – By default, Kubernetes captures the standard output and standard error of each container. You can view them with `kubectl logs <pod>`.

Logging Challenges

- Pods are ephemeral – logs disappear when Pods are deleted.

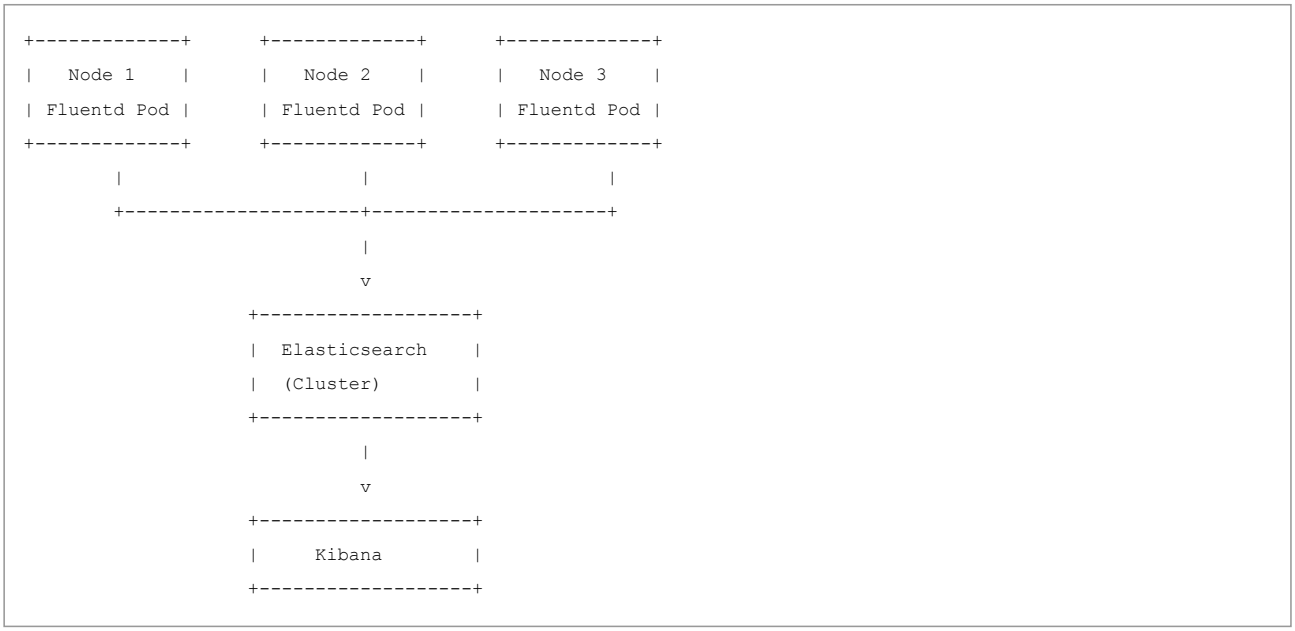
- Aggregating logs from many Pods across nodes is hard.

Cluster-Level Logging – Solutions that collect, store, and query logs centrally. Common stacks:

- **EFK/ELK**: Elasticsearch (storage & search), Fluentd/Logstash (log forwarder), Kibana (visualization).
- **Loki**: A Prometheus-inspired log aggregation system (Grafana Loki) designed to be cost-effective.

Fluentd DaemonSet – A common pattern is to run Fluentd as a DaemonSet on each node. It tails container log files and forwards them to Elasticsearch or other backends.

Diagram: EFK Stack on Kubernetes



10:45 – 12:30 – Hands-on Lab 1: Prometheus & Grafana

Exercise 1.1 – Install Prometheus Stack with Helm

We'll use the `kube-prometheus-stack` chart, which includes Prometheus, Grafana, Alertmanager, and several exporters.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
kubectl create namespace monitoring
helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring
```

This may take a minute. Check the pods:

```
kubectl get pods -n monitoring -w
```

Wait until all are running.

Exercise 1.2 – Access Prometheus UI

Port-forward to the Prometheus service:

```
kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090 &
```

Open your browser to `http://localhost:9090`. In the **Graph** tab, try a query:

```
container_cpu_usage_seconds_total{namespace="default"}
```

Click **Execute**. You'll see time series for containers in the default namespace.

Exercise 1.3 – Access Grafana

Get the Grafana admin password (by default, it's `prom-operator`; but we can retrieve the secret):

```
kubectl get secret -n monitoring prometheus-grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo
```

Port-forward to Grafana:

```
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80 &
```

Open `http://localhost:3000`. Login with username `admin` and the password from above.

Explore the pre-configured dashboards (click **Dashboards** → **Browse**). There are dashboards for Kubernetes cluster monitoring, nodes, pods, etc.

Exercise 1.4 – Create Custom Grafana Dashboard

Let's create a simple dashboard showing CPU and memory usage for the `default` namespace.

1. Click **+** → **Create** → **Dashboard**.
2. Click **Add new panel**.
3. In the query editor, select **Prometheus** data source.
4. Enter the following PromQL query for CPU:

   ```
   sum(rate(container_cpu_usage_seconds_total{namespace="default", container!=""}[5m])) by (pod)
   ```

5. In the **Panel options**, set title to "CPU Usage per Pod (default ns)".
6. Click **Apply**.
7. Add another panel for memory:

   ```
   sum(container_memory_usage_bytes{namespace="default", container!=""}) by (pod)
   ```

8. Arrange panels and save the dashboard (give it a name, e.g., "Default Namespace Metrics").

Exercise 1.5 – Set Up an Alert

Create a `PrometheusRule` resource to alert when any pod in the default namespace uses more than 0.8 CPU cores for 2 minutes.

Save the following YAML as `cpu-alert.yaml`:

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: cpu-alert
  namespace: monitoring
spec:
  groups:
  - name: cpu
    rules:
    - alert: HighCPUUsage
      expr: sum(rate(container_cpu_usage_seconds_total{namespace="default"}[5m])) by (pod) > 0.8
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: "High CPU usage detected on {{ $labels.pod }}"
```

Apply it:

```
kubectl apply -f cpu-alert.yaml
```

This rule will appear in Prometheus (under **Alerts**) and can trigger Alertmanager. (Alertmanager configuration is out of scope for today.)

13:30 – 15:00 – Conceptual Session: Autoscaling & Best Practices

2.1 Scaling in Kubernetes

Manual Scaling

You can manually change the number of replicas in a Deployment:

```
kubectl scale deployment my-deployment --replicas=5
```

Horizontal Pod Autoscaling (HPA)

HPA automatically adjusts the number of Pod replicas based on observed metrics (CPU, memory, or custom metrics). It works with the **Metrics Server** (which provides resource usage metrics).

HPA Algorithm

Desired replicas = ceil[current replicas * (current metric value / desired metric value)]

Example: If current CPU usage is 200m, desired is 100m, and there are 2 replicas, then desired = ceil[2 * (200/100)] = 4.

HPA YAML Example:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Vertical Pod Autoscaling (VPA)

VPA adjusts the CPU/memory **requests** and **limits** of containers based on usage, rather than the number of replicas. It's useful for stateful applications where scaling out is not possible. VPA can recommend values or automatically update Pods (which may require restart).

2.2 HPA Mechanics

Metrics Server

HPA requires the **Metrics Server** to be installed in the cluster. It collects resource metrics from the kubelet and exposes them via the Metrics API. Without it, `kubectl top` and HPA won't work.

Target Average Utilization

You specify a target average utilization (percentage of the requested CPU). HPA computes the current utilization and adjusts replicas accordingly.

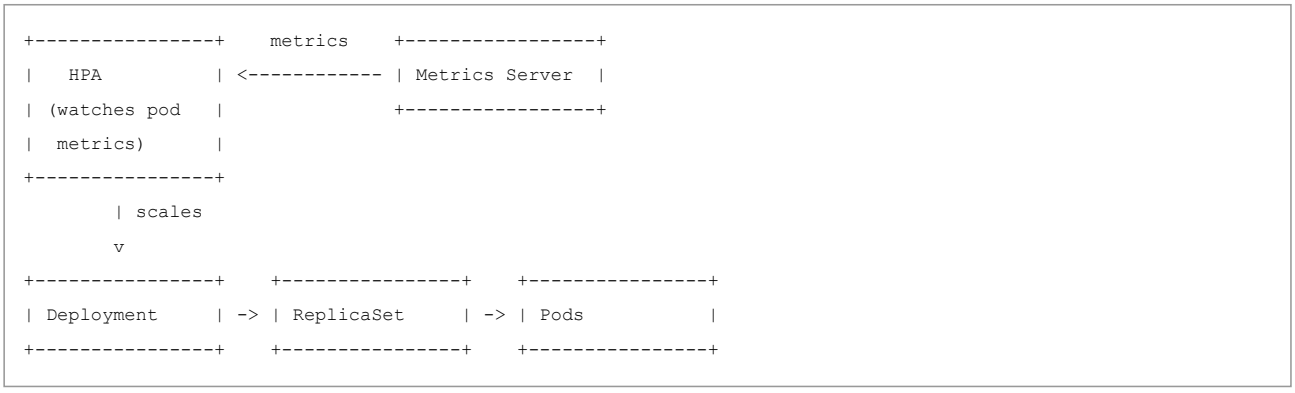
Stabilization Window

To avoid thrashing (frequent scaling), HPA has a **stabilization window** (configurable) that limits how fast it can scale down.

Scaling Behavior

- Scale up is more aggressive than scale down (by default, scale up can happen every 15 seconds, scale down every 5 minutes).
- You can customise these with `behavior` field in HPA v2.

Diagram: HPA Workflow



2.3 Best Practices for Production

Resource Requests and Limits

Always set CPU and memory requests/limits for every container. This helps the scheduler make better decisions and prevents noisy neighbours.

- **requests**: What the container is guaranteed.
- **limits**: Hard cap; container cannot exceed.

Example:

```
resources:
  requests:
    memory: "256Mi"
    cpu: "200m"
  limits:
    memory: "512Mi"
    cpu: "500m"
```

Readiness and Liveness Probes

- **livenessProbe**: Checks if the container is alive; if it fails, kubelet restarts the container.
- **readinessProbe**: Checks if the container is ready to serve traffic; if it fails, the Pod is removed from Service endpoints.

Probe Types: `httpGet`, `tcpSocket`, `exec`.

Example:

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
```

PodDisruptionBudget (PDB)

PDB limits the number of Pods of a replicated application that can be voluntarily disrupted (e.g., during node drain). It ensures high availability during cluster maintenance.

Example:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: myapp-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: myapp
```

Namespace Isolation

Use namespaces to separate environments (dev, staging, prod) and apply ResourceQuotas and NetworkPolicies to enforce boundaries.

Network Policies

Network policies control traffic flow between Pods and from outside. By default, all Pods can communicate. With a NetworkPolicy, you can restrict ingress/egress based on labels and ports. Requires a CNI that supports NetworkPolicy (e.g., Calico, Cilium).

2.4 Advanced Orchestration Features

- **StatefulSets**: For stateful applications that require stable network identities and persistent storage (e.g., databases).
- **DaemonSets**: Ensures that a copy of a Pod runs on all (or some) nodes (e.g., log collectors, monitoring agents).
- **Jobs**: Runs a Pod to completion (batch task).
- **CronJobs**: Runs Jobs on a schedule.

15:15 – 16:45 – Hands-on Lab 2: Autoscaling & Production Readiness

Exercise 2.1 – Deploy an Application with Resource Limits

Create a file `php-apache.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-apache
spec:
  selector:
    matchLabels:
      run: php-apache
  replicas: 1
  template:
    metadata:
      labels:
        run: php-apache
    spec:
      containers:
      - name: php-apache
        image: registry.k8s.io/hpa-example
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: 200m
            memory: 256Mi
          limits:
            cpu: 500m
            memory: 512Mi
```

Apply:

```
kubectl apply -f php-apache.yaml
```

Exercise 2.2 – Install Metrics Server

Minikube does not install Metrics Server by default. Install it:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Metrics Server may need a modification to work with Minikube's self-signed certificates. Edit the deployment:

```
kubectl edit deployment metrics-server -n kube-system
```

Add the following argument to the container's args section:

```
- --kubelet-insecure-tls
```

Save and exit. Wait for the pod to restart:

```
kubectl get pods -n kube-system -w | grep metrics-server
```

Verify metrics are available:

```
kubectl top nodes
kubectl top pods
```

Exercise 2.3 – Create HPA

Create an HPA that targets 50% CPU utilization:

```
kubectl autoscale deployment php-apache --cpu-percent=50 --min=1 --max=10
```

Check the HPA:

```
kubectl get hpa -w
```

Initially, it will show `<unknown>/50%` until metrics are collected.

Exercise 2.4 – Generate Load

Open another terminal and run a load generator:

```
kubectl run load-generator --image=busybox -- /bin/sh -c "while true; do wget -q -O- http://php-apache; done"
```

Now watch the HPA in the first terminal:

```
kubectl get hpa -w
```

You should see the CPU usage increase, and after a short while, the HPA will scale up the replicas (to 2, 3, ...). The load generator may need a minute to register.

To stop the load generator, delete it:

```
kubectl delete pod load-generator
```

Exercise 2.5 – Liveness and Readiness Probes

Update the `php-apache` deployment to include probes. First, edit the deployment:

```
kubectl edit deployment php-apache
```

Add the following under `containers` (after `resources`):

```
livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 5
```

Save and exit. The deployment will roll out new pods.

Check the pod status:

```
kubectl get pods
```

Describe a pod to see probe details:

```
kubectl describe pod <pod-name>
```

You'll see events about the probes.

Exercise 2.6 – PodDisruptionBudget

Create a PDB to ensure at least 2 replicas are always available (since our app currently has 1, the PDB won't be effective until we scale). Scale to 3 first:

```
kubectl scale deployment php-apache --replicas=3
```

Now create `pdb.yaml`:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: php-apache-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      run: php-apache
```

Apply:

```
kubectl apply -f pdb.yaml
```

To test, you can try draining a node (but with Minikube single node, it’s not meaningful). You can simulate a voluntary disruption by evicting a pod:

```
kubectl drain minikube --ignore-daemonsets
```

But that would evict everything. Instead, just observe the PDB status:

```
kubectl get pdb
```

You'll see `ALLOWED DISRUPTIONS` – the number of pods that can be voluntarily disrupted.

Exercise 2.7 – Cleanup Everything

```
kubectl delete deployment php-apache
kubectl delete hpa php-apache
kubectl delete pdb php-apache-pdb
helm uninstall prometheus -n monitoring
kubectl delete namespace monitoring
kubectl delete -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Stop Minikube (optional):

```
minikube stop
```

Daily Recap & Troubleshooting

Key Takeaways

- **Prometheus** scrapes metrics; **Grafana** visualises them. Together they form a powerful monitoring stack.
- **HPA** automatically scales replicas based on CPU/memory usage (or custom metrics).
- Always set **resource requests/limits** to ensure predictable scheduling and avoid resource starvation.
- **Probes** (liveness, readiness) make your applications self-healing and traffic-ready.
- **PodDisruptionBudget** protects workloads during voluntary disruptions.
- For logging, tools like Fluentd and Elasticsearch (EFK) aggregate logs.

Common Issues & Solutions

Problem	Likely Cause	Solution
`kubectl top` returns error	Metrics Server not installed or misconfigured.	Check Metrics Server pods; add `--kubelet-insecure-tls` if using Minikube.
HPA shows `<unknown>` for metrics	Metrics not available yet, or HPA targeting wrong metric.	Wait; check `kubectl get --raw /apis/metrics.k8s.io/v1beta1/pods`.
Grafana login fails	Wrong password.	Retrieve password from secret (see Exercise 1.3).
Pod stuck in `CrashLoopBackOff`	Application crashes, or probes failing.	Check logs with `kubectl logs`; adjust probes.
Prometheus targets not scraped	ServiceMonitor missing or labels mismatch.	Verify ServiceMonitor configuration and service labels.
PDB prevents eviction	PDB `minAvailable` is too high.	Reduce `minAvailable` or increase replicas.

Useful Commands Recap

Command	Description
`kubectl top node` / `kubectl top pod`	Show resource usage.
`kubectl get hpa -w`	Watch HPA status.
`kubectl describe pod <name>`	Show pod details, including probe events.
`kubectl get --raw /apis/metrics.k8s.io/v1beta1/pods`	Raw metrics API access.
`helm list -n <namespace>`	List Helm releases.

Command	Description
`kubectl get pdb`	List PodDisruptionBudgets.

Connection to Previous Modules

- **Scripting (Days 36–40)**: Automating monitoring setup and scaling tests.
- **Infrastructure (Days 45–47)**: Resource management, network policies relate to infrastructure concepts.
- **Security (Days 49–50)**: RBAC used to secure monitoring components; network policies for micro-segmentation.
- **DevSecOps (Days 26–30)**: CI/CD pipelines can trigger HPA tests, integrate monitoring alerts.

Next Steps & Certification Path

- **Certifications**: CKA (Certified Kubernetes Administrator), CKAD (Certified Kubernetes Application Developer), KCNA (Kubernetes and Cloud Native Associate).
- **Further Learning**:
  - Service Mesh (Istio, Linkerd) for advanced traffic management and observability.
  - GitOps (ArgoCD, Flux) for declarative continuous delivery.
  - Custom Metrics for HPA (e.g., based on queue length).
  - Explore CNCF landscape for other projects.

Congratulations! You have completed the 4-day Containers & Kubernetes workshop. You now possess foundational to intermediate skills in containerization and orchestration. Keep practicing, explore more advanced topics, and consider pursuing certification. Happy Kuberneting!